

Inigo Quilez :: articles :: 3D distance and gradient functions - 2025



Intro

Knowing the gradient of an SDF is very useful. The gradient for an SDF at any point always has length 1 (since the SDF measures distances), so it's also a normal to the isosurface passing through that point. This gradient also points in the direction of the closest point, and as a normal, is useful for lighting, aligning objects to surfaces, etc etc. Of course, one can use **central differences to compute gradients**, but that has a high computational cost (requires 4 or 6 taps), and can get tricky to tweak its epsilon values to always give good results. Automatic differentiation is another way to compute gradients, and works well with SDFs of course, but we can do even better.

In this page you'll find a list of primitives and their analytic gradients, where lots of terms are reused between the SDF computation and its gradient computation, making them very cheap to evaluate. The list is incomplete, but when you don't find a particular primitive in it you can very easily reach out to the **2D SDF Gradients list**, and extend the primitive you want to 3D through thru extrusion or revolution and carry its gradient/normal with it.

Lastly, note that all the code listed here comes with a link to a realtime online demo in Shadertoy. In fact, this public playlist contains all the Shadertoy examples: <https://www.shadertoy.com/playlist/cXccR2>, so don't miss that out.

Primitives

All primitives are centered at the origin; you will have to transform the point to get arbitrarily rotated, translated and scaled objects, and transform the gradient back accordingly (remember not to use non uniform scales since such transforms do not preserve distances). All SDFs below return the actual distance in the $.x$ component, and its partial derivatives with respect to x and y in the $.y$, $.z$ and $.w$ components. In other words,

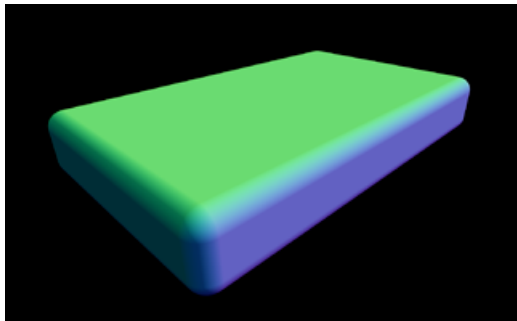
$$.x = f(p)$$

$$.y = \partial f(p) / \partial x$$

$$.z = \partial f(p) / \partial y$$

$$.w = \partial f(p) / \partial z$$

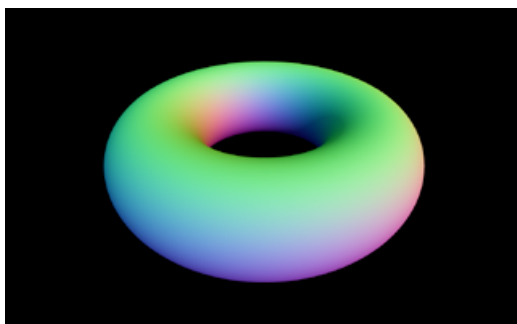
$$.yzw = \nabla f(p) \text{ with } \|\nabla f(p)\| = 1$$



Box

<https://www.shadertoy.com/view/WfdyD4>

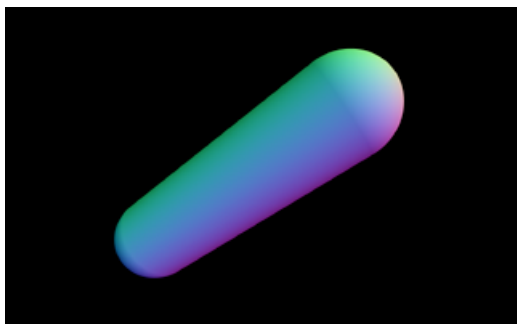
```
vec4 sdgBox( in vec3 p, in vec3 b, in float r )
{
    vec3 w = abs(p)-(b-r);
    float g = max(w.x,max(w.y,w.z));
    vec3 q = max(w,0.0);
    float l = length(q);
    vec4 f = (g>0.0)?vec4(l, q/l) :
                vec4(g, w.x==g?1.0:0.0,
                    w.y==g?1.0:0.0,
                    w.z==g?1.0:0.0);
    return vec4(f.x-r, f.yzw*sign(p));
}
```



Torus

<https://www.shadertoy.com/view/wtcfzM>

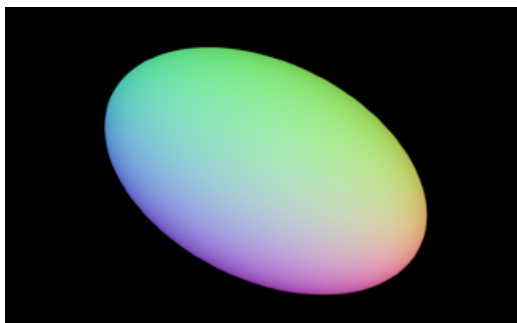
```
vec4 sdgTorus( in vec3 p, in float ra, in float rb )
{
    float h = length(p.xz);
    return vec4( length(vec2(h-ra,p.y))-rb,
                 normalize(p*vec3(h-ra,h,h-ra)) );
}
```



Segment

<https://www.shadertoy.com/view/WttfR7>

```
vec4 sdgSegment( in vec3 p, in vec3 a, in vec3 b, in float r )
{
    vec3 ba = b-a;
    vec3 pa = p-a;
    float h = clamp( dot(pa,ba)/dot(ba,ba), 0.0, 1.0 );
    vec3 q = pa-h*ba;
    float d = length(q);
    return vec4(d-r,q/d);
}
```

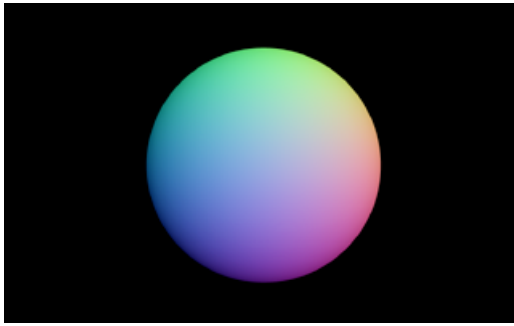


Ellipsoid

<https://www.shadertoy.com/view/flXyRS>

Note: exact gradient but innexact distance

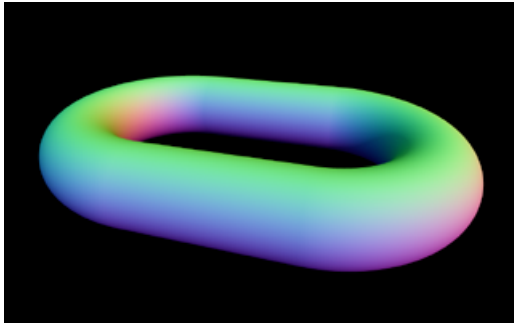
```
vec4 sdgEllipsoid( in vec3 p, in vec3 r )
{
    p /= r; float k0 = sqrt(dot(p,p));
    p /= r; float k1 = inversesqrt(dot(p,p));
    return vec4( k0*(k0-1.0)*k1, p*k1 );
}
```



Sphere

<https://www.shadertoy.com/view/wcccW7>

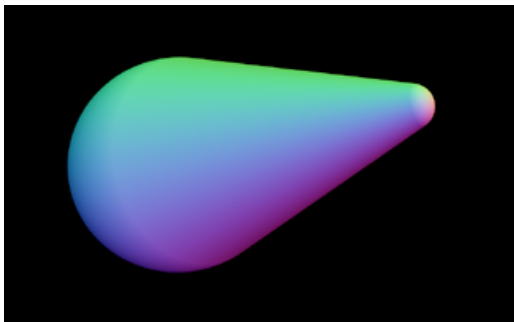
```
vec4 sdgSphere( in vec3, in float r )
{
    float l = length(p);
    return vec4(1-r, p/l);
}
```



Link

<https://www.shadertoy.com/view/wc3yD7>

```
vec4 sdgLink( in vec3 p, in float le, in float r1, in float r2 )
{
    vec3 q = vec3( p.x, p.y-clamp(p.y,-le,le), p.z );
    float w = length(q.xy);
    float l = length(vec2(w-r1,q.z));
    return vec4(1-r2, (q-vec3(r1*q.xy/w,0.0))/l);
}
```



Rounded Cone

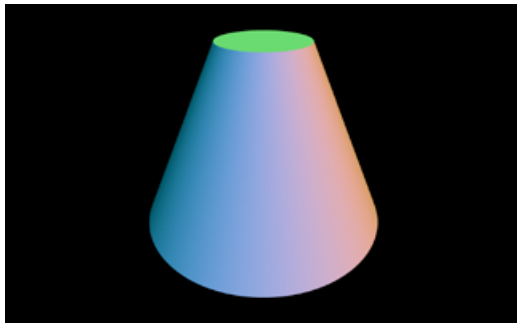
<https://www.shadertoy.com/view/tfccD7>

```

vec4 sdgRoundCone( vec3 p, vec3 a, vec3 b, float r1, float r2 )
{
    vec3 ba = b - a;
    float l2 = dot(ba,ba);
    float rr = r1 - r2;
    float a2 = l2 - rr*rr;
    float il2 = 1.0/l2;

    vec3 pa = p - a;
    vec3 pb = p - b;
    float y = dot(pa,ba);
    float z = y-l2; //dot(pb,ba)
    float x2 = l2*dot(pa,pa)-y*y;
    float y2 = y*y;
    float z2 = z*z;
    float k = sign(rr)*rr*rr*x2;
    if( sign(z)*a2*z2>k ) { float w=sqrt(il2*(x2+z2));
                           return vec4(w-r2,pb/w); }
    if( sign(y)*a2*y2<k ) { float w=sqrt(il2*(x2+y2));
                           return vec4(w-r1,pa/w); }
    { float w=sqrt(x2*a2);
      return vec4((w+y*rr)*il2-r1,
                  il2*(rr*ba+a2*(pa*l2-y*ba)/w)); }
}

```



Vertical Capped Cone

<https://www.shadertoy.com/view/3ctcW7>

```

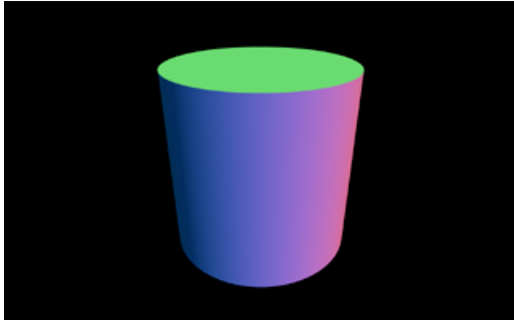
vec4 sdgCappedCone( in vec3 p, in float he, in float r1, in float r2 )
{
    vec2 k = vec2(r2-r1,2.0*he);
    float m = dot(k,k);
    float l = length(p.xz);
    vec2 q = vec2(r2-l, he-p.y);
    vec2 a = vec2(1-min(l,p.y<0.0?r1:r2), abs(p.y)-he);
    vec2 b = k*clamp(dot(q,k)/m,0.0,1.0) - q;
    float s = (b.x<0.0 && a.y<0.0) ? -1.0 : 1.0;
}

```

```

float la = dot(a,a);
float lb = dot(b,b);
return (la<lb)?vec4(s*sqrt(la), 0.0, sign(p.y), 0.0 ) :
        vec4(s*sqrt(lb),vec3(k.y*p.xz/l,-k.x).xzy/sqrt(m));
}

```



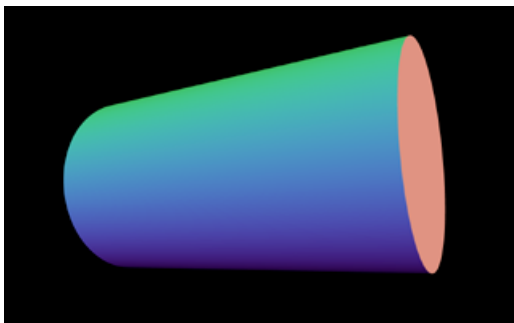
Vertical Cylinder

<https://www.shadertoy.com/view/3fcyz2>

```

vec4 sdgCylinder( in vec3 p, in float he, in float r )
{
    float l = length(p.xz);
    vec2 e = vec2(l-r,abs(p.y)-he/2.0);
    vec2 h = max(e,0.0);
    float f = length(h);
    float g = max(e.x,e.y);
    vec3 du = vec3(p.x/l,0.0,p.z/l);
    vec3 dv = vec3(0.0,p.y<0.0?-1.0:1.0,0.0);
    return (g<=0.0) ? vec4( g, (e.x>e.y)?du:dv ) :
        vec4( f, (h.x*du+h.y*dv)/f );
}

```



Cylinder

<https://www.shadertoy.com/view/3fcyz2>

```
vec4 sdgCylinder( in vec3 p, in vec3 a, in vec3 b, in float r )
{
    vec3 ba = (b-a)*0.5;
    vec3 ce = (b+a)*0.5;
    float l = length(ba);
    vec3 d = ba/l;

    vec3 q = p-ce;
    float v = dot(q,d);
    vec3 u = q-v*d;
    float k = length(u); // k = sqrt(dot(q,q)-v*v);
    vec2 e = vec2(k-r,abs(v)-1);
    vec2 h = max(e,0.0);
    float f = length(h);
    float g = max(e.x,e.y);
    vec3 du = u/k;
    vec3 dv = v<0.0?-d:d;
    return (g<=0.0) ? vec4(g, (e.x>e.y)?du:dv):
                vec4(f, (h.x*du+h.y*dv)/f);
}
```

inigo quilez - learning computer graphics since 1994